

Tag 3

Fehler finden, Texte zerschneiden & selbst recherchieren

Python-Grundlagen-Schulung

Was du heute lernst

1. Kurzes Aufwärmen mit dem bisher Gelernten
2. Fehlermeldungen **lesen** und **reparieren** (Debugging)
3. Einzelne Zeichen aus einem Text herausschneiden (**Slicing**)
4. Wie du selbst nach Python-Lösungen **recherchierst**

Teil 1

Aufwärmen

Vier kurze Wiederholungsaufgaben

Bevor es weitergeht, frischst du kurz auf,
was du bisher gelernt hast:

- Alter berechnen
- Minuten in Stunden umrechnen (`//` und `%`)
- Rabatt berechnen
- Terabyte in Gigabyte umrechnen

Alle vier Aufgaben findest du im **Colab-Notebook**.

Teil 2

Der Code-Sanitäter

Fehler lesen und reparieren

Die wichtigste Debugging-Regel

**Lies die Fehlermeldung
von UNTEN nach OBEN.**

Die **letzte Zeile** verrät dir meistens,
was wirklich schiefgelaufen ist.

Fehler 1: Text minus Zahl (TypeError)

```
offene_tickets = input("Wie viele offene Tickets? ")
geloeste_tickets = 5

verbleibend = offene_tickets - geloeste_tickets
print("Es verbleiben: " + verbleibend)
```

✗ **TypeError:** `input()` liefert immer **Text** –
Text minus Zahl geht nicht!

So wird Fehler 1 repariert

```
offene_tickets = int(input("Wie viele offene Tickets? "))
geloeste_tickets = 5

verbleibend = offene_tickets - geloeste_tickets
print(f"Es verbleiben: {verbleibend}")
```

✓ `int(...)` wandelt die Eingabe in eine Zahl um.

✓ Ein f-String kann Text und Zahl problemlos mischen.

Fehler 2: Klammer und Tippfehler

```
srv_name = "SRV-Backup"  
status_aktiv = True  
  
print(f"Server {srv_Name} hat den Status: {status_aktiv}")
```

✗ Erst: fehlende schließende Klammer)

✗ Dann: `srv_Name` mit großem N – aber die Variable heißt `srv_name` mit **kleinem** n!

Wichtig zu wissen

Python unterscheidet **streng** zwischen
Groß- und Kleinschreibung.

`srv_name` und `srv_Name` sind für Python
zwei völlig unterschiedliche Variablen!

So wird Fehler 2 repariert

```
srv_name = "SRV-Backup"  
status_aktiv = True  
  
print(f"Server {srv_name} hat den Status: {status_aktiv}")
```

✓ Klammer geschlossen, Schreibweise korrigiert.

Fehler 3: Dezimalzahl mit int() (ValueError)

```
freier_speicher = int(input("Wie viel GB freier Speicher? "))  
bedarf = 1.2  
rest = freier_speicher - bedarf  
print(f"Restlicher Speicher: {rest} GB")
```

Eingabe: 2.5 → **✗ ValueError**

`int()` kann keine Zahl mit Komma/Punkt verarbeiten!

So wird Fehler 3 repariert

```
freier_speicher = float(input("Wie viel GB freier Speicher? "))  
bedarf = 1.2  
rest = freier_speicher - bedarf  
print(f"Restlicher Speicher: {rest} GB")
```

✓ float(...) statt int(...) –
erlaubt Dezimalzahlen.

Fehler 4: Division durch Null

```
gesamtspeicher_gb = 500  
aktive_server = 0  
  
last_pro_server = gesamtspeicher_gb / aktive_server
```

✗ ZeroDivisionError

Eine Division durch Null ist mathematisch
nicht definiert – Python bricht deshalb ab.

So wird Fehler 4 repariert

```
gesamtspeicher_gb = 500
aktive_server = 2 # darf nicht 0 sein!

last_pro_server = gesamtspeicher_gb / aktive_server
print(f"Jeder Server erhält {last_pro_server} GB.")
```

✓ Der Nenner darf **niemals 0** sein.

Im echten IT-Alltag: genau das passiert,
wenn z. B. gerade kein Server aktiv ist!

Teil 3

Texte gezielt zerschneiden

String-Slicing

Ein String ist eine Kette aus Zeichen

Zeichen:	P	y	t	h	o	n
Index+:	0	1	2	3	4	5
Index-:	-6	-5	-4	-3	-2	-1

Jedes einzelne Zeichen hat eine **Nummer (Index)** –
gezählt ab **0**, von links **und** von rechts.

Ein einzelnes Zeichen holen

```
wort = "Python"

print(wort[0])    # P  (erstes Zeichen)
print(wort[-1])   # n  (letztes Zeichen)
```

Eckige Klammern `[...]` mit einer **Zahl**
holen dir genau **ein** Zeichen an dieser Position.

Einen ganzen Bereich ausschneiden

```
wort = "Python"  
print(wort[0:3])    # Pyt
```

`[start:stop]` schneidet einen **Bereich** aus.

⚠ Wichtig: **stop** gehört nicht mehr dazu!

`[0:3]` heißt: Index 0, 1, 2 – **nicht** Index 3.

Bild: Slicing wie ein Lineal

	P	y	t	h	o	n
0	1	2	3	4	5	6

Die Zahlen stehen **zwischen** den Buchstaben.

`wort[0:3]` schneidet zwischen Position 0 und 3 –
das ergibt **"Pyt"**.

Schrittweite: Zeichen überspringen

```
wort = "Python"

print(wort[::2])    # Pto    (jedes zweite Zeichen)
print(wort[::-1])   # nohtyP (rückwärts!)
```

[start:stop:step] – der dritte Wert ist die **Schrittweite**.

`::-1` ist ein beliebter Trick, um einen kompletten String rückwärts auszugeben.

Merksatz

**[start:stop] schneidet aus –
stop selbst ist NICHT mehr dabei.**



Kurze Pause

Teil 4

String-Methoden & selbst recherchieren

Was ist eine Methode?

Eine Methode ist ein **fertiger Befehl**,
den du direkt an einen String anhängst.

```
text = "Hallo Python"  
print(text.upper())    # HALLO PYTHON  
print(text.lower())    # hallo python
```

Der Punkt `.` verbindet den String mit der Methode.

Die wichtigste Fähigkeit heute: selbst suchen!

Niemand kann sich **alle** Methoden merken –
auch erfahrene Programmierer/innen googeln ständig.

Das ist völlig normal und keine Schwäche.

Die Such-Formel

python string [was ich tun möchte]

Beispiel: "python string remove spaces"

→ führt dich direkt zur Methode `.strip()`

So gehst du vor

1. Überlege: **Was** will ich mit dem Text machen?
(z. B. "Leerzeichen entfernen")
2. Formuliere das auf **Englisch**, ganz simpel
3. Google: `python string [dein Ziel]`
4. Meistens landest du direkt bei der passenden Methode

Teil 5

Jetzt du: der große Übungs-Slam

Teil A: Slicing-Aufgaben

Ländercode aus Servernamen extrahieren

Dateiendung mit negativem Index bestimmen

Mittleren Teil einer Seriennummer herausschneiden

Ein Wort rückwärts ausgeben

Jeden zweiten Buchstaben herausfiltern

Teil B: Recherche-Aufgaben

Diese Aufgaben verraten dir **nicht** direkt die Methode – du sollst sie selbst googeln!

- Leerzeichen entfernen
- Prüfen, ob eine Datei auf `.exe` endet
- Zeichen in einem Pfad ersetzen
- Vorkommen eines Wortes zählen
- Prüfen, ob ein String nur Ziffern enthält
- Eine Zeile am Semikolon zerlegen

Ein Beispiel zum Warmwerden

```
server_name = "DE-MUN-SRV01"  
laendercode = server_name[0:2]  
print(laendercode)    # DE
```

Genau das Prinzip, das du gerade
Schritt für Schritt kennengelernt hast.

Was du heute gelernt hast

- ✓ Fehlermeldungen von **unten nach oben** lesen
- ✓ `[start:stop]` schneidet Text aus – `stop` zählt nicht mehr mit
- ✓ `[::-1]` dreht einen String um, `[::2]` überspringt Zeichen
- ✓ Bei unbekannten Methoden: einfach **googeln** – das ist normal!

Morgen: Coding-Arena & Einstieg in Listen 🎉